

Experiment No. 1

Aim:

Cryptography in Blockchain, Merkle Root Tree Hash

Theory:

Objectives:

To explore fundamental blockchain concepts involving cryptographic hashing, Proof-of-Work, nonce generation, and Merkle Trees.

Tasks Performed:

1. SHA-256 hash generation.
2. Target hash generation using a nonce.
3. Cryptographic puzzle solving to find the Golden Nonce.
4. Merkle Tree construction and generation of the Merkle Root.

Tools and Libraries:

- Google Colab
- Python
- Hashlib

1. Cryptographic Hash Functions in Blockchain

A cryptographic hash function converts data into a fixed-length hash. **SHA-256** is commonly used in blockchain for data integrity, transaction security, and Proof-of-Work. A small change in input produces a completely different hash.

2. What is a Merkle Tree?

A **Merkle Tree** is a tree structure that organizes transaction hashes. Hashes are combined and hashed repeatedly until one final hash, called the **Merkle Root**, is obtained.

3. What is a Cryptographic Puzzle and Golden Nonce?

A cryptographic puzzle requires finding a value that satisfies a specific hash condition. In Proof-of-Work, miners change the **nonce** until the hash meets the required difficulty, such as starting with zeros. The nonce that produces the valid hash is called the **Golden Nonce**.

4. How Does a Merkle Tree Work?

1. Hash each transaction.
2. Combine two hashes.
3. Hash the combined value.
4. Repeat until one hash remains.
5. The final hash is the **Merkle Root**.

Experiment No. 1**5. Benefits of Merkle Tree**

- Ensures data integrity.
- Detects data tampering.
- Provides fast transaction verification.
- Reduces the amount of data needed for verification.
- Efficient for large datasets.

6. Use Cases of Merkle Tree

Merkle Trees are mainly used in **blockchain transaction verification, Bitcoin, data integrity checking, and distributed systems**

Code:

Program #1: Python program that uses the hashlib library to create the hash of a given string

```
import hashlib
# Take input from user
data = input("Enter the data: ")
# Generate SHA-256 hash
hash_value = hashlib.sha256(data.encode()).hexdigest()
print("Original Data:", data)
print("SHA-256 Hash:", hash_value)
```

Output:

```
Enter the data: Hello Dipesh
Original Data: Hello Dipesh
SHA-256 Hash: 2c17f26f3d8702a9f8dbc09ee59e2ffa375cc957f4243ac9e5eafc6cdbafdecc
```

Program #2: Program to generate required target hash with input string and nonce

```
import hashlib
data = input("Enter the data: ")
nonce = int(input("Enter the nonce: "))
combined_data = data + str(nonce)
hash_value = hashlib.sha256(combined_data.encode()).hexdigest()
print("Data:", data)
print("Nonce:", nonce)
print("Combined Data:", combined_data)
print("Hash:", hash_value)
```

Experiment No. 1**Output:**

Enter the data: Dipesh
Enter the nonce: 10
Data: Dipesh
Nonce: 10
Combined Data: Dipesh10
Hash: dd71dd6f06dd32225ab6f0053e14bcbacfe61e9841e0abd987be00f65f108740

Program #3: Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib
data = input("Enter the data: ")
difficulty = int(input("Enter difficulty (number of leading zeros): "))
target = "0" * difficulty
nonce = 0
while True:
    text = data + str(nonce)
    hash_value = hashlib.sha256(text.encode()).hexdigest()
    if hash_value.startswith(target):
        break
    nonce += 1
print("\nCryptographic Puzzle Solved!")
print("Data:", data)
print("Golden Nonce:", nonce)
print("Hash:", hash_value)
print("Target:", target)
```

Output:

Enter the data: Dipesh
Enter difficulty (number of leading zeros): 4

Cryptographic Puzzle Solved!
Data: Dipesh
Golden Nonce: 79753
Hash: 000098bed95f5924393d22ca7b4bf5930834055487ba001b6c8cd60d38f36c09
Target: 0000

Experiment No. 1

Program 4: Generating Merkle Tree for given set of Transactions

```
import hashlib
def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None
    # Hash each transaction first
    transactions = [
        hashlib.sha256(transaction.encode('utf-8')).hexdigest()
        for transaction in transactions
    ]
    print("=== Transaction Hashes ===")
    for i, transaction_hash in enumerate(transactions):
        print(f"T{i+1}: {transaction_hash}")
    print()
    level = 1
    # Construct Merkle Tree
    while len(transactions) > 1:
        # If odd number of hashes, duplicate the last hash
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])
        new_transactions = []
        print(f"=== Merkle Tree Level {level} ===")
        for i in range(0, len(transactions), 2):
            left = transactions[i]
            right = transactions[i + 1]
            combined = left + right
            hash_combined = hashlib.sha256(
                combined.encode('utf-8')
            ).hexdigest()
            print(f"Left Hash : {left}")
            print(f"Right Hash : {right}")
            print(f"Combined : {combined}")
            print(f"Parent Hash : {hash_combined}")
            print("-" * 70)
            new_transactions.append(hash_combined)
        transactions = new_transactions
        level += 1
    return transactions[0]
# Given transactions
transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $500",
```

Experiment No. 1

```

"Dave -> Eve : $100",
"Eve -> Alice : $300",
"Roo -> Bob : $50"
]
# Generate Merkle Root
merkle_root = build_merkle_tree(transactions)
print("\n=== FINAL MERKLE ROOT ===")
print("Merkle Root:", merkle_root)

```

Output:

=== Transaction Hashes ===

```

T1: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
T2: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
T3: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
T4: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
T5: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac

```

=== Merkle Tree Level 1 ===

```

Left Hash  : f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
Right Hash  : 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
Combined   :
f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760768890d3b58d4
d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
Parent Hash : 471608d3d6f2a642ff3a84dfaceaebb1c271a9aed5b37a82d61c4784558e80e9
-----

```

```

Left Hash  : 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
Right Hash  : cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
Combined   :
43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890dacba341d7965fff7
3181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
Parent Hash : a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
-----

```

```

Left Hash  : cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
Right Hash  : cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
Combined   :
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58accfe1dc26c7c3e26
c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
Parent Hash : 314ca746b3dfe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
-----

```

=== Merkle Tree Level 2 ===

```

Left Hash  : 471608d3d6f2a642ff3a84dfaceaebb1c271a9aed5b37a82d61c4784558e80e9
Right Hash  : a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
Combined   :
471608d3d6f2a642ff3a84dfaceaebb1c271a9aed5b37a82d61c4784558e80e9a61c504d72e6ce
69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
Parent Hash : ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
-----

```

Experiment No. 1

Left Hash : 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Right Hash : 314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Combined :
314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e314ca746b3dafa63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Parent Hash : 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

=== Merkle Tree Level 3 ===

Left Hash : ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
Right Hash : 6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45
Combined :
ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45
Parent Hash : 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

=== FINAL MERKLE ROOT ===

Merkle Root: 96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Conclusion:

The experiment successfully demonstrated **SHA-256 hashing, nonce-based cryptographic puzzles, and Merkle Tree construction**. It showed how hashing and Merkle Roots help ensure **data integrity, security, and tamper detection** in blockchain transactions.